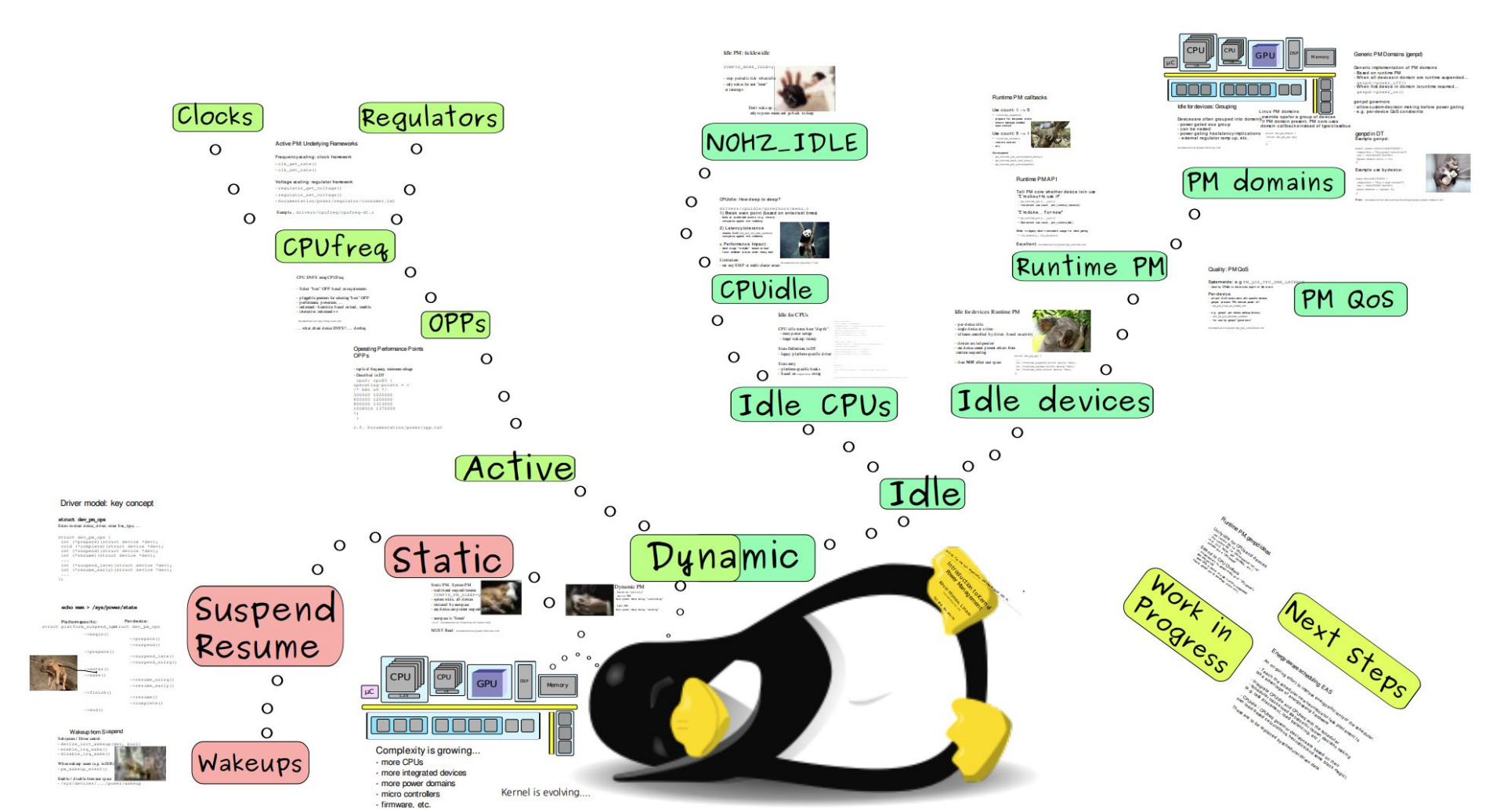
Google

Monitoring power saving states

Power MC, Linux Plumbers Conference 2023

Sven van Ashbrook

Stanislaw Kardach



http://events17.linuxfoundation.org/sites/events/files/slides/Intro_Kernel_PM.pdf

Idle

NOHZ_IDLE

Idle PM: tickless idle

CONFIG_NOHZ_IDLE=y

- stop periodic tick when idle
- only wakes for next "event" or interrupt



Don't wake up...
only to press snooze and go back to sleep

CPUidle

CPUidle: How deep to sleep?

drivers/cpuidle/governors/menu.c

1) Break even point (based on enter/exit times)
- looks at predictable events (e.g. timers)
- compares against min residency

2) Latency tolerance
- checks QoS (PM_QOS_CPU_DMA_LATENCY)
- compares against min residency

3) Performance impact
- black magic "multiplier" based on load
- favor shallower states under heavy load

Limitations:
- not very SMP or multi-cluster aware



Documentation/cpuidle/*.txt

Idle CPUs

Idle for CPUs

CPU idle states have "depth"
- more power savings
- longer wakeup latency

State Definitions in DT
- legacy: platform-specific driver

State entry
- platform-specific hooks
- based on compat-like string

Runtime PM

Runtime PM: callbacks

Use count: 1 --> 0
- --> runtime_suspend()
- prepare for low-power state
- ensure wakeups enabled
- save context

Use count: 0 --> 1
- --> runtime_resume()
- restore context
- etc.



Autosuspend
- pm_runtime_set_autosuspend_delay()
- pm_runtime_mark_last_busy()
- pm_runtime_put_autosuspend()

Runtime PM API

Tell PM core whether device is in use

"I'm about to use it"
- pm_runtime_get () , _async()
- Increment use count, pm_runtime_resume()

"I'm done... for now"
- pm_runtime_put () , _async()
- Decrement use count, pm_runtime_idle()

Similar to legacy clock framework usage for clock gating
- clk_enable(), clk_disable()

Excellent: Documentation/power/pm_runtime.txt

Idle devices

Idle for devices: Runtime PM

- per-device idle
- single device at a time
- idleness controlled by driver, based on activity

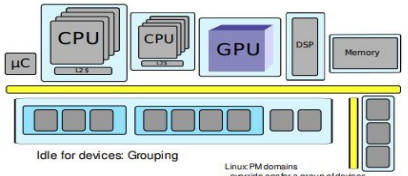
- devices are *independent*
- one device cannot prevent others from runtime suspending

- does **NOT** affect user space



```
struct dev_pm_ops {
  ...
  int (*runtime_suspend) (struct device *dev);
  int (*runtime_resume) (struct device *dev);
  int (*runtime_idle) (struct device *dev);
};
```

PM domains



Idle for devices: Grouping

Devicesare often grouped into domains
- power gated as a group
- can be nested
- power gating has latency implications
- external regulator ramp up, etc.

Documentation/power/devices.txt

Linux PM domains
- override ops for a group of devices
- if PM domain present, PM core uses domain callbacks instead of type/class/bus

```
struct dev_pm_domain {
  struct dev_pm_ops ops;
  ...
};
```

Generic PM Domains (genpd)

Generic implementation of PM domains
- Based on runtime PM
- When all devices in domain are runtime suspended... genpd->power_off()
- When first device in domain is runtime resumed... genpd->power_on()

genpd governors
- allow custom decision making before power gating
- e.g. per-device QoS constraints

genpd in DT

Example genpd:

```
power: power-controller@12340000 {
  compatible = "foo,power-controller";
  reg = <0x12340000 0x1000>;
  #power-domain-cells = <1>;
};
```

Example use by device:

```
leaky-device@12350000 {
  compatible = "foo,i-leak-current";
  reg = <0x12350000 0x1000>;
  power-domains = <&power 0>;
};
```

From: Documentation/devicetree/bindings/power/power-domain.txt



PM QoS

Quality: PM QoS

System-wide: e.g. PM_QOS_CPU_DMA_LATENCY
- Used by CPUidle to determine depth of idle state

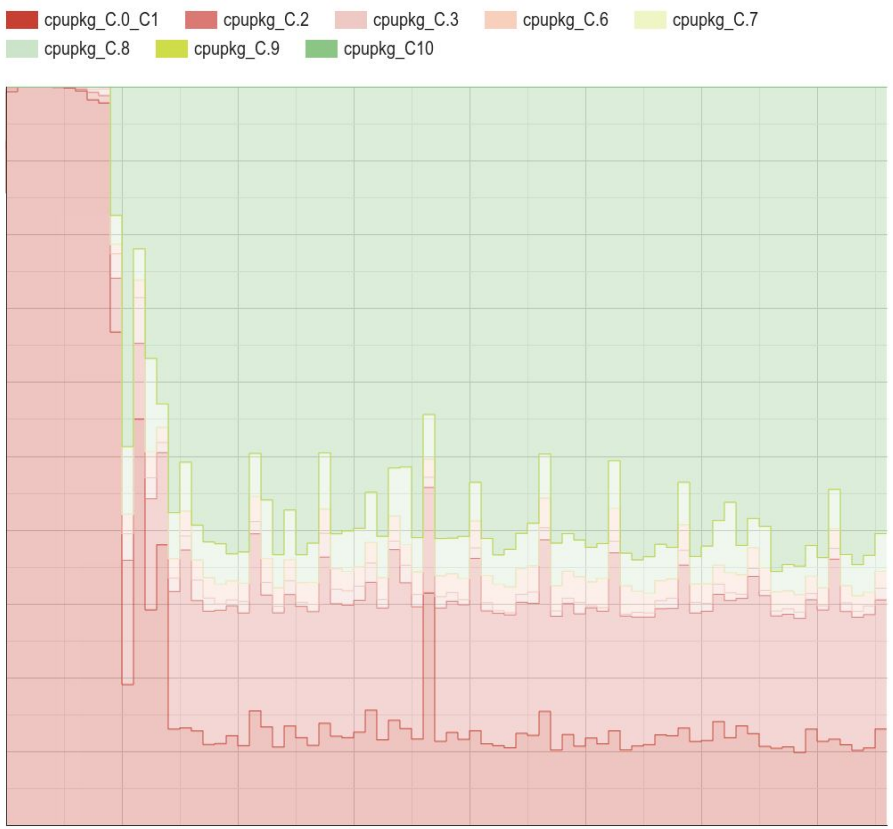
Per-device
- attach QoS constraints with specific devices
- genpd: prevent PM domain power off
- PM_QOS_FLAG_NO_POWER_OFF

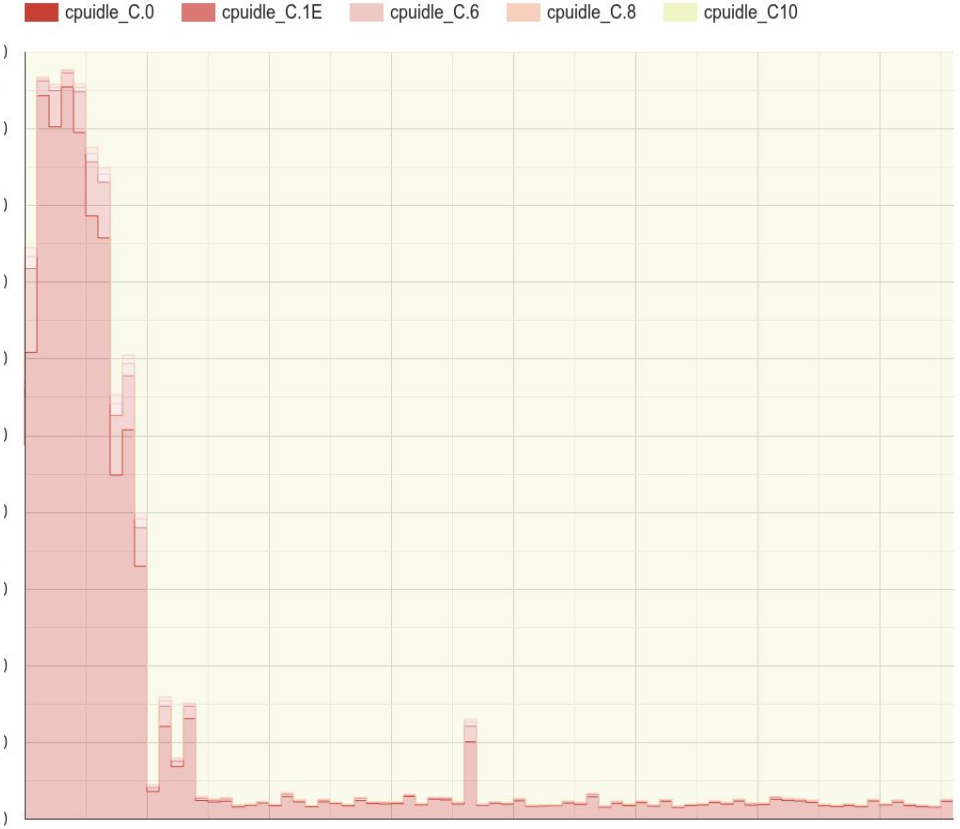
- e.g. genpd: per-device wakeup latency
- DEV_PM_QOS_RESUME_LATENCY
- for use by genpd "governors"

Documentation/power/pm_qos_interface.txt

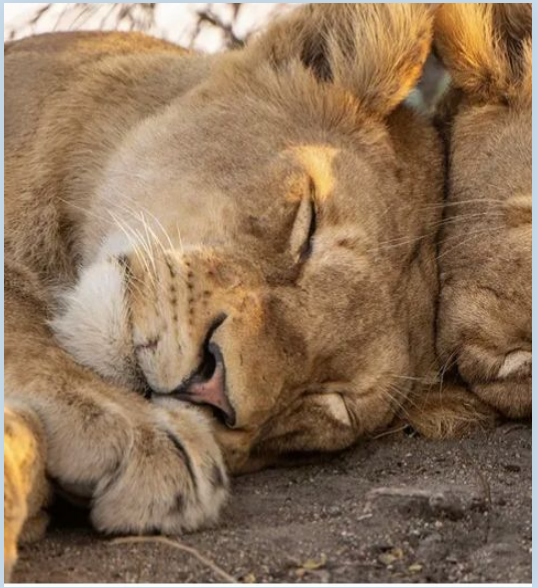
Proprietary + Confidential

We monitor and track dynamic idle states





cpuidle states



- ARM: WFI, cpu off/sleep
- X86: c-states C0, C2, C8, C10, ...

cpuidle driver provides residency stats in sysfs.
Great! Except:

- Lack of atomicity (~100 sysfs nodes to read)
- Incorrect due to state demotion: kernel ignorant of true state

module idle states

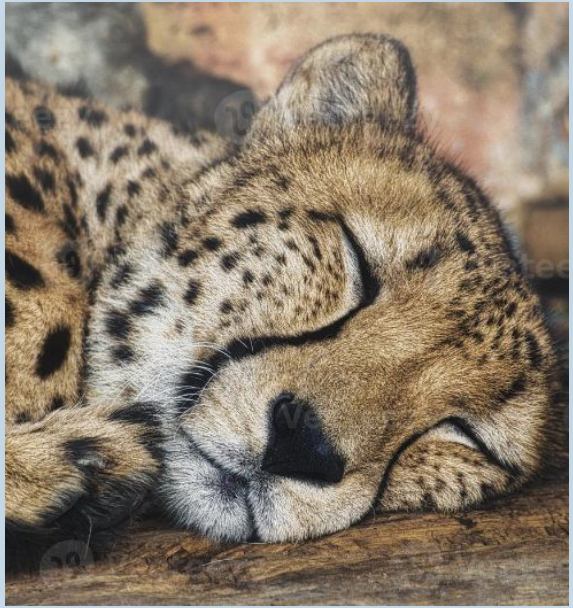


- ARM: cluster sleep/off, fits nicely in cpuidle !
- X86: package c-states PC0, PC2, PC8, PC10, ...

Residency stats through debugfs or vendor tools.
(mostly, see ACPI LPIT)
Not so great. Also:

- debugfs formats “differ” between x86 vendors, kernel versions
- PC0 missing from debugfs, invites accumulating systematic error
- vendor registers invite duplicate userspace implementations

SoC dynamic idle states



- ARM: aosd, cxds. Fits nicely in genpd !
- X86: S0ix currently S0i2.0 - S0i3.4

Residency stats through debugfs or vendor tools.
(mostly, see ACPI LPIT)
Not so great. Also:

- debugfs formats differ between x86 vendors, kernel versions
- Total time may be missing from debugfs, invites accumulating systematic error
- vendor registers invite duplicate userspace implementations

Google

Static system idle - s2idle



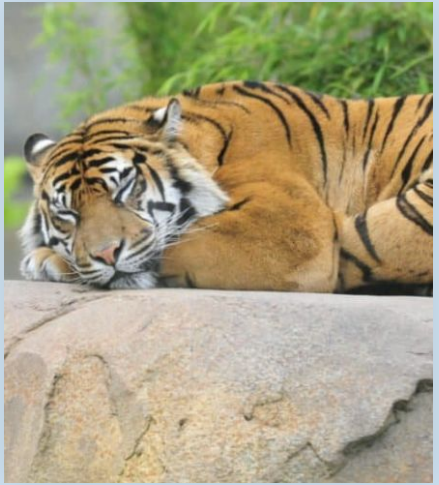
s2idle repurposes dynamic idle states for static system idle. (Possibly) comparable power savings and lower static resume latency.

Residency stats historically through patchwork of vendor specific paths. Mario Limonciello landed improvement - thank you !

However, gaps remain:

- How deep is the deepest reached idle state?
- Zero s2idle residency is common failure mode - what prevented the system from getting there?

Discuss - the way forward



- Are changes to the current patchwork desired?
- Generic, standardized idle stats ?
- Include topology or hierarchy?
- Special relationship to s2idle (like some cpuidle stats have) ?
- Integrated debug for s2idle entry failing requirements?
- How does LPIT fit in?

Slides under CC-BY-SA 3.0